

Algorithmics	Student information	Date	Number of session
	UO: 307423	17/03	3
	Surname: Miranda Martín	 Escuela de Ingeniería Informática Universidad de Oviedo	
	Name: Mariana Patricia		



Activity 1. [Divide and Conquer by subtraction]

After analyzing the complexity of the three previous classes, you are not asked to create the time tables, but to reason whether the times match the theoretical complexity of each algorithm.

For which value of n do the Subtraction1 and Subtraction2 classes stop producing execution times (we abort the algorithm because it exceeds 1 minute)?

Subtraction1.java and Subtraction2.java classes overflow when the problem size grows to several thousand.

Subtraction1.java stops after the problem size grows more than 32 768, whereas Subtraction2.java stops after the problem size grows more than 16384.

Why does that happen?

As the message shown in the output console indicates, there is an StackOverflow exception when executing these classes for big problem sizes.

How many years would it take to complete the Subtraction3 execution for $n=80$? Explain your reasoning.

After executing Subtraction3, I chose $n_1 = 27$, which takes $t_1 = 5370$ milliseconds.

$$t_2 = t_1 \times \frac{2^{80}}{2^{27}} = t_1 \times 2^{53} \approx 4.3 \times 10^{19} \text{ ms}$$

$$4.3 \times 10^{19} \text{ ms} \times \frac{1 \text{ year}}{3.15 \times 10^{10}} = 1.37 \times 10^9 \text{ years}$$

It will approximately take 1.3 billion years.

Algorithmics	Student information	Date	Number of session
	UO: 307423	17/03	3
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Implement a Subtraction4.java class with complexity $O(n^3)$ and then fill in a table showing the time (in milliseconds) for $n=100, 200, 400, 800, \dots$ (until OoT).

N	Time (ms)
100	0
200	2
400	2
800	0
1600	4
3200	29
6400	259
12800	15510

Implement a Subtraction5.java class with complexity $O(3^{n/2})$ and then fill in a table showing the time (in milliseconds) for $n=30, 32, 34, 36, \dots$ (until OoT).

N	Time (ms)
30	453
32	1288
34	3947
36	11709
38	35505
40	OoT
42	OoT

How many years would it take to complete the Subtraction5 execution for $n=80$? Explain your reasoning. Complexity $O(3^{n/2})$

After executing Subtraction5, I chose $n_1 = 36$, which takes $t_1 = 11709$ milliseconds.

$$t_2 = t_1 \times \frac{3^{\frac{80}{2}}}{3^{\frac{36}{2}}} = t_1 \times 3^{22} \approx 3.6 \times 10^{14} \text{ ms}$$

$$3.6 \times 10^{14} \text{ ms} \times \frac{1 \text{ year}}{3.15 \times 10^{10} \text{ ms}} = 1.1 \times 10^4 \text{ years}$$

It will approximately take 11 000 years.

Algorithmics	Student information	Date	Number of session
	UO: 307423	17/03	3
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 2. [Divide and conquer by division]

After analyzing the complexity of the three previous classes, you are not asked to create the time tables, but to reason whether the times match the theoretical complexity of each algorithm.

They do, in fact, match their theoretical complexity, as expected.

Implement a Division4.java class with complexity $O(n^2)$ (with $a < b^k$) and then fill in a table showing the time (in milliseconds) for $n=1000, 2000, 4000, 8000, \dots$ (up to OoT).

N	Time (ms)
100	0
200	0
400	1
800	5
1600	19
3200	74
6400	307
12800	1224
25600	5748
51200	26519

Implement a Division5.java class with complexity $O(n^2)$ (with $a > b^k$) and then fill in a table showing the time (in milliseconds) for $n=1000, 2000, 4000, 8000, \dots$ (up to OoT).

N	Time (ms)
100	1
200	2
400	9
800	37
1600	148
3200	585
6400	2369
12800	12760
25600	50774

Activity 3. [Two basic examples]

Algorithmics	Student information	Date	Number of session
	UO: 307423	17/03	3
	Surname: Miranda Martín		
	Name: Mariana Patricia		

VectorSum1.java solves the simple problem of adding the elements of a vector in three different ways, and VectorSum2.java measures times of these three algorithms for different sizes of the problem.

Fibonacci1.java solves the problem of calculating the Fibonacci number of order n in four different ways, and Fibonacci2.java measures times of these four algorithms for different sizes of the problem.

After analyzing the complexity of the various algorithms within the two classes, executing them and after putting the times obtained in a table, compare the efficiency of each algorithm.

Algorithmics	Student information	Date	Number of session
	UO: 307423	17/03	3
	Surname: Miranda Martín		
	Name: Mariana Patricia		

NTIMES = 50000			
N	vectorSum1	vectorSum2	vectorSum3
3	0,0001	0,0001	0,0001
6	0,0001	0,0002	0,0002
12	0,0001	0,0003	0,0005
24	0,0002	0,0005	0,001
48	0,0004	0,0009	0,002
96	0,0007	0,0017	0,004
192	0,0014	0,0034	0,0079
384	0,0027	0,0067	0,016
768	0,0053	0,0137	0,0323
1536	0,0107	0,0271	0,0646
3072	0,0214	0,0543	0,1295
6144	0,0425	0,1093	0,2587
12288	0,0841	OoT	0,5171
24576	0,1676	OoT	1,032
49152	0,3349	OoT	2,0673
98304	0,6804	OoT	OoT

VectorSum1 is clearly the most efficient out of the three methods. It follows a linear complexity, i.e., $O(n)$ and implements an iterative solution.

VectorSum2 also follows a linear complexity. However, the space complexity due to the call stack is also $O(n)$, which causes it to be slower.

VectorSum3 also follows a linear complexity. In contrast of the previous two implementations, this method has a Divide and Conquer approach, in which the array is split by half every time the method is called. Although it is also a recursive implementation, it is faster than VectorSum1 due to the fact that its space complexity is $O(\log n)$ instead.

Algorithmics	Student information	Date	Number of session
	UO: 307423	17/03	3
	Surname: Miranda Martín		
	Name: Mariana Patricia		

NTIMES = 50000				
N	fibonacci1	fibonacci2	fibonacci3	fibonacci4
10	0,0001	0,0002	0,0002	0,0033
15	0,0002	0,0003	0,0003	0,0358
20	0,0002	0,0004	0,0004	0,3987
25	0,0002	0,0004	0,0005	4,4779
30	0,0003	0,0005	0,0005	OoT
35	0,0003	0,0006	0,0006	OoT
40	0,0003	0,0007	0,0007	OoT
45	0,0004	0,0007	0,0008	OoT
50	0,0004	0,0008	0,0008	OoT
55	0,0004	0,0008	0,001	OoT
60	0,0005	0,0009	0,001	OoT

Fibonacci1 is the most efficient out of the four methods. It follows a linear complexity and uses just one loop.

Fibonacci2 is also an iterative implementation, but it uses an array which makes it less efficient in terms of memory than Fibonacci1, though it also follows a linear complexity.

Fibonacci3 is a recursive implementation, which makes exactly n recursive calls. Therefore, it also follows a linear complexity.

Fibonacci4 is also a recursive implementation. However, as you can see from the table, it is extremely inefficient. It follows a $O(2^n)$ complexity.

Algorithmics	Student information	Date	Number of session
	UO: 307423	17/03	3
	Surname: Miranda Martín		
	Name: Mariana Patricia		

Activity 5. [Closest Points]

FIRST PART

Solve the problem using a trivial algorithm with a class called `PointsTrivial.java`, which calculates the solution by determining the distance between every pair of distinct points, thus revealing the desired pair of points and their minimum distance.

Create a class called `PointsTrivialTimes.java` that, after randomly generating n points, calculates the execution time of the trivial algorithm:

N	time (ms)
1024	23
2048	31
4096	38
8192	92
16384	314
32768	1253
65536	4965
131072	20230
262144	OoT
524288	OoT
1048576	OoT

Consider whether the times obtained are consistent with the complexity of the algorithm.

There is indeed a consistency with the data. As the method implemented has a quadratic theoretical complexity, i.e., $O(n^2)$, by doubling the input each iteration, the execution times should be for times bigger as it grows exponentially.

Algorithmics	Student information	Date	Number of session
	UO: 307423	17/03	3
	Surname: Miranda Martín		
	Name: Mariana Patricia		

SECOND PART

Solve the problem using a Divide and Conquer algorithm with a class PointsDaC.java that, by dividing the problem into subproblems, allows you to determine the desired pair of points and their minimum distance.

Create a class PointsDaCTimes.java that, after randomly generating n points, calculates the execution time of the new algorithm:

N	time (ms)
1024	37
2048	4
4096	4
8192	7
16384	25
32768	46
65536	143
131072	210
262144	217
524288	531
1048576	1067
2097152	2534
4194304	5718
8388608	12307
16777216	28917

Consider whether the times obtained are consistent with the complexity of the algorithm.

There is indeed a consistency with the data. As the method implemented has a theoretical complexity of $O(n * \log(n))$ and as you can see, it corresponds with the data.